

RS001 Week 7 — English Worksheet

Topic: Stopping Mistakes — Validating Input · **Course:** Text Adventure (Python) · **Time:** about 45 minutes

This week you check whether an answer is **allowed** before you act on it. `in` and `not in` test if a value is inside a list of valid choices. If the answer is wrong, you give a second chance — and if it is still wrong, you fall back to a safe default.

```
valid = ["left", "right", "back"]
choice = game.ask("Where to?").strip().lower()
if choice in valid:
    game.say(f"You go {choice}.")
else:
    game.say("You cannot go that way.")
```

1 · Predict 🧠

Read each piece of code. Before you run it in your head, write what you think happens.

```
valid = ["left", "right"]
choice = "up"
game.say(choice in valid)
```

`in` answers True or False. Which one is printed?


```
valid = ["left", "right"]
choice = "left"
if choice not in valid:
    game.say("not allowed")
else:
    game.say("allowed")
```

Which message appears?


```
valid = ["left", "right"]
choice = game.ask("Where?").strip().lower()
if choice not in valid:
    choice = "left"
game.say(choice)
```

The player types `"up"`. What value ends up in `choice` ?

2 · Spot the Bug 🐛

Each block was meant to do something, but it is broken. Read what it is **supposed** to do, fix it, then explain why the original was wrong.

Bug A — This should warn the player when their choice is **not** valid. Right now the warning shows for valid answers instead.

```
valid = ["left", "right"]
choice = game.ask("Where?").strip().lower()
if choice in valid:
    game.say("That is not a real direction.")
```

Hint: the warning belongs with `not in`, not `in`.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug B — On a wrong answer, the player should get **one more try**. Right now a wrong answer just gives up with no second chance.

```
valid = ["left", "right", "back"]
choice = game.ask("Where to?").strip().lower()
if choice not in valid:
    game.say("Not a valid way.")
    game.say(f"Final: {choice}")
```

Hint: after the warning, ask again and store the new answer.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug C — If the answer is still wrong after the retry, the game should pick a safe **default** so it never breaks. Right now a wrong answer is used as-is.

```
valid = ["left", "right"]
choice = game.ask("Where to?").strip().lower()
if choice not in valid:
    choice = game.ask("left or right?").strip().lower()
game.say(f"You move {choice}.")
```

Hint: add one more check that swaps in a default like `"left"` if it is still invalid.

Write the fixed code:

Why was it wrong? Why does your fix work?

3 · Validate a Choice

Open your game. Take your path choice and validate it against a list of valid directions. If the player gets it wrong, give one extra try, then fall back to a safe default so the game keeps going.

When it works, send a **photo or video**, then explain what you did. Use these sentence starters — write 4 to 6 sentences total.

First, I listed the valid choices, which were ...

I used `in` (or `not in`) to ...

When the player typed something wrong, the game ...

My safe default was ...

One tricky moment was when ...

If I had more time, I would ...

4 · Record Your Walkthrough 🎥

Take a video on your phone (or a parent's phone) while your game runs. Teach it like the viewer has never coded. Try to use these words: **valid**, **in**, **not in**, **retry**, **default**.

1. Run your game and type a valid answer.
2. Run it again and type something silly on purpose.
3. Show the retry, then the safe default kicking in.
4. Read your validation line out loud and explain what `not in` checks.

Write what you will say in your video. Plan it here first.

Submit ✓

Send this worksheet + your walkthrough video to teacher on KakaoTalk.